GYANMANJARI INNOVATIVE UNIVERSITY

Gyanmanjari Institute of Technology – Faculty of Engineering &Technology

Tree Mastery

Red-Black Trees, B-Trees, Splay Trees and 2-3 Trees: properties, operations and performance

Student Name	Kothari Tarj Mehulbhai	Enrollment No.	260205020001
Subject	AdvancedDataStructure (METCE11502)	Semester	1 (M.Tech)
Activity	ALA 2	Marks	10

1. Aim

To explore the **properties and operations** of Red-Black trees, B-trees, Splay trees and 2-3 trees; to implement these structures and perform **insertions and deletions**; and to **analyse their performance**.All four structures were implemented in Python, verified with automated tests, and measured. The results shown are real program output.

2. Introduction

A plain binary search tree (BST) can become a chain when keys arrive in sorted order, giving O(n) operations. Balanced and self-adjusting trees prevent this:

Tree	Idea	Height / cost guarantee
Red-Black tree	BSTwithacolourbitpernode;colourruleskeepthe longest path at most twice the shortest.	Height $\leq 2 \cdot \log_2(n+1)$; search, insert, delete: O(log n) worst case; at most 2 rotations per insert and 3 per delete.
B-tree (order m)	Multiwaysearchtree;anodehasuptom−1keysand m children. All leaves are at the same depth; overfull nodes split, underfull nodes borrow or merge. Designed for disks/pages.	Height$O(\log_{m/2} n)$;alloperations$O(\log n)$ worst case.
2-3 tree	B-tree of order 3: every internal node has 2 or 3 children(1or2keys);allleavesatthesamedepth.	$\log_3(n+1) \leq \text{height} \leq \log_2(n+1)$;O(logn) worst case.
Splay tree	BSTwithnobalanceinformation:everyaccessed node is moved to the root by *splaying* (rotations).	O(log n) *amortized* per operation; a singleoperationcancostO(n).Adaptsto access patterns.

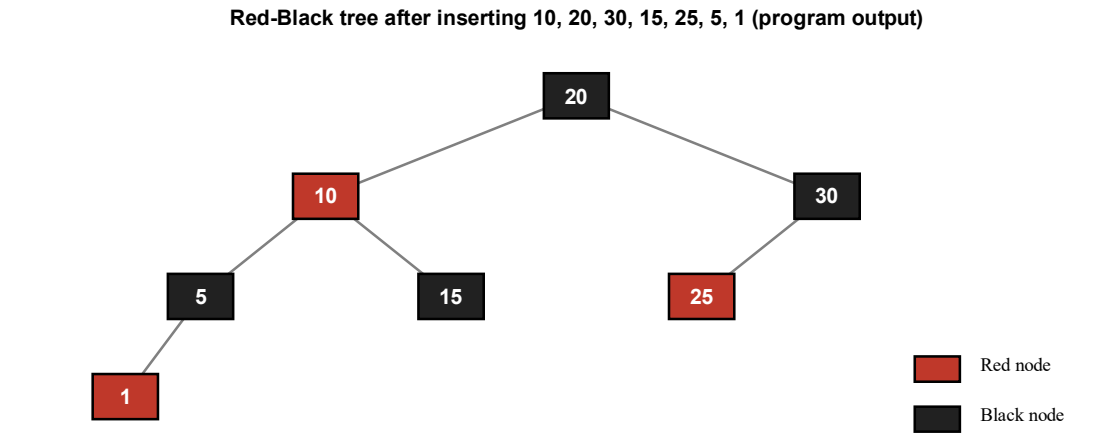


Figure 1: A valid red-black tree: root black, no red node has a red child, every path has 3 black nodes (counting the NIL leaf).

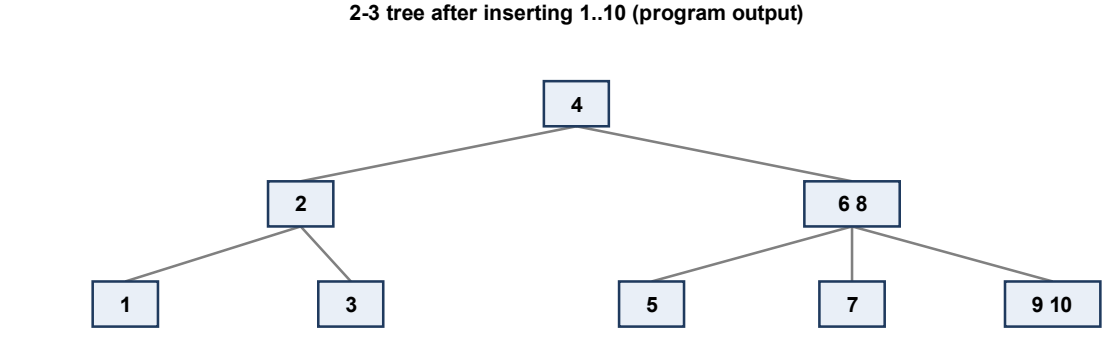


Figure 2: All leaves are on the same level; each node has 1 or 2 keys. Splits (5 in total) pushed middle keys upward.

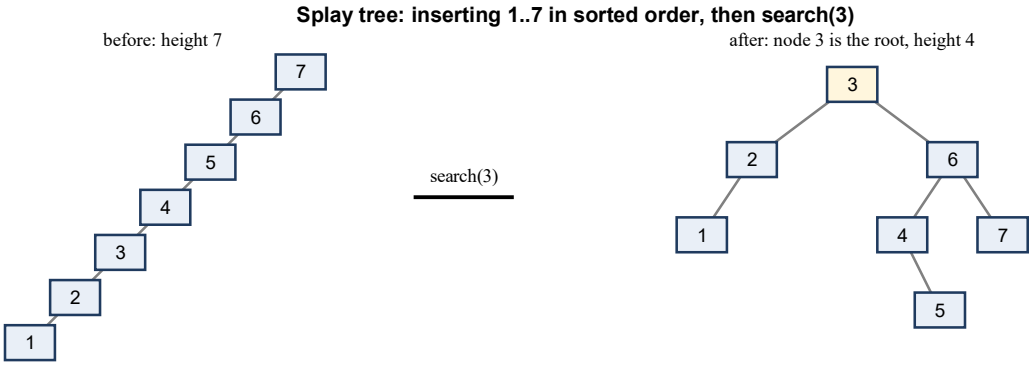


Figure 3: Splaying moves the accessed key to the root and roughly halves the depth of the nodes on the access path.

3. Red-Black Tree

Properties

- Every node is red or black; the root is black; every NIL leaf is black.
- A red node never has a red child (no two reds in a row).
- EverypathfromanodetoitsdescendantNILleavescontainsthesamenumberofblacknodes (black-height).
- Consequence:the longest path is at most twice the shortest, so height $\leq 2 \cdot \log_2(n+1)$.

Operations

Case(newnodezis red)	Condition	Action
Insert case 1	Parent and uncle are red	Recolourparentanduncleblack,grandparent red; continue from grandparent.
Insert case 2	Parentred,uncleblack,zisaninner child (triangle)	Rotateparentsothatzbecomesanouterchild; go to case 3.
Insert case 3	Parentred,uncleblack,zisanouter child (line)	Recolourparentblack,grandparentred;rotate the grandparent.
Delete (x = node that replacestheremoved one)	Ablacknodewasremoved,sox carries an 'extra black'	Fix-up loop with sibling w:
Delete case 1	Sibling w is red	Recolour,rotateparent;nowwisblack(cases 2–4).
Delete case 2	w black with two black children	Recolourwred;movetheextrablackuptothe parent.
Delete case 3	wblack,nearchildred,farchild black	Recolourandrotatewsothatthefarchildisred (case 4).
Delete case 4	w black, far child red	Recolour,rotatetheparent,extrablackis absorbed; stop.

Source code

```
RED,BLACK=True,False


classRBNode:
    slots=("key","color","left","right","parent")

    def init(self, key, color=RED):
        self.key, self.color = key, color
        self.left=self.right=self.parent=None


classRedBlackTree:
    definit(self):
        self.NIL=RBNode(None,BLACK)
        self.NIL.left = self.NIL.right = self.NIL.parent = self.NIL
        self.root = self.NIL
        self.rotations=0

    def_left_rotate(self,x): y
        = x.right
        x.right=y.left
        ify.leftisnotself.NIL:
            y.left.parent = x
        y.parent=x.parent
        ifx.parentisself.NIL:
            self.root = y
        elifxisx.parent.left:
            x.parent.left = y
        else:
            x.parent.right=y
        y.left = x
        x.parent = y
        self.rotations+=1

    def_right_rotate(self,x): y
        = x.left
        x.left=y.right
        ify.rightisnotself.NIL:
            y.right.parent = x
        y.parent=x.parent
        ifx.parentisself.NIL:
            self.root = y
        elifxisx.parent.right:
            x.parent.right = y
        else:
            x.parent.left=y
        y.right = x
        x.parent = y
        self.rotations+=1

    defsearch(self,key): x
        = self.root
        while x is not self.NIL and key != x.key:x
            = x.left if key < x.key else x.right
        returnxifxisnotself.NILelseNone

    definsert(self,key):
        z=RBNode(key,RED)
        z.left=z.right=self.NIL y, x
        = self.NIL, self.root while
        x is not self.NIL:
            y=x
            ifkey==x.key:
                return False                        #noduplicates x
            = x.left if key < x.key else x.right
        z.parent=y
           ifyisself.NIL:
            self.root = z
           elifkey<y.key:
            y.left=z else:
            y.right = z
        self._insert_fixup(z)
        return True
```

```
def _insert_fixup(self, z):
    while z.parent.color == RED:
        ifz.parentisz.parent.parent.left:
            u=z.parent.parent.right                          #uncle
            if u.color == RED:                               #case1:recolour
                z.parent.color = u.color = BLACK
                z.parent.parent.color=RED z
                = z.parent.parent
            else:
                if z is z.parent.right:                      #case2:triangle->line z =
                    z.parent
                    self._left_rotate(z)
                z.parent.color = BLACK                       # case 3: rotate grandparent
                z.parent.parent.color = RED
                self._right_rotate(z.parent.parent)
        else:                                                #mirrorimage
            u=z.parent.parent.left if
            u.color == RED:
                z.parent.color = u.color = BLACK
                z.parent.parent.color = RED
                z=z.parent.parent
            else:
                ifzisz.parent.left: z
                    = z.parent
                    self._right_rotate(z)
                z.parent.color = BLACK
                z.parent.parent.color = RED
                self._left_rotate(z.parent.parent)
    self.root.color=BLACK

def _transplant(self, u, v):
    ifu.parentisself.NIL:
        self.root=v
    elifuisu.parent.left:
        u.parent.left = v
    else:
        u.parent.right=v
    v.parent = u.parent

def_minimum(self,x):
    while x.left is not self.NIL:
        x = x.left
    returnx

defdelete(self,key):
    z=self.search(key) if
    z is None:
        returnFalse
    y, y_orig_color = z, z.color
    if z.left is self.NIL:
        x = z.right
        self._transplant(z, z.right)
    elifz.rightisself.NIL: x
        = z.left
        self._transplant(z, z.left)
    else:
        y = self._minimum(z.right)                           #in-ordersuccessor
        y_orig_color = y.color
        x=y.right
        ify.parentisz:
            x.parent = y
        else:
            self._transplant(y, y.right)
            y.right = z.right
            y.right.parent = y
        self._transplant(z,y)
        y.left = z.left
        y.left.parent = y
        y.color = z.color
    if y_orig_color == BLACK:                                #ablacknodevanished
        self._delete_fixup(x)
    returnTrue

def_delete_fixup(self,x):
    while x is not self.root and x.color == BLACK:
        if x is x.parent.left:
            w=x.parent.right                                 #sibling
```

```
            ifw.color==RED:                                  #case1
                w.color, x.parent.color = BLACK,
                REDself._left_rotate(x.parent)
                w=x.parent.right
            if w.left.color == BLACK and w.right.color == BLACK:    #case2
                w.color = RED
                x=x.parent else:
                if w.right.color == BLACK:                  #case3
                    w.left.color, w.color = BLACK, RED
                    self._right_rotate(w)
                    w=x.parent.right
                w.color, x.parent.color = x.parent.color, BLACK    #case4
                w.right.color = BLACK
                self._left_rotate(x.parent)
                x = self.root
        else:                                                #mirrorimage
            w = x.parent.left
            ifw.color==RED:
                w.color, x.parent.color = BLACK,
                REDself._right_rotate(x.parent)
                w=x.parent.left
            if w.right.color == BLACK and w.left.color == BLACK:
                w.color = RED
                x=x.parent else:
                if w.left.color == BLACK:w.right.color,
                    w.color = BLACK, RED
                    self._left_rotate(w)
                    w=x.parent.left
                w.color, x.parent.color = x.parent.color, BLACK
                w.left.color = BLACK
                self._right_rotate(x.parent)
                x=self.root
    x.color = BLACK

# ---- helpers for testing / analysis ----
def inorder(self):
    out, st, x = [], [], self.root
    whilestorxisnotself.NIL:
        whilexisnotself.NIL:
            st.append(x)
            x=x.left x
        = st.pop()
        out.append(x.key)
        x = x.right
    returnout

defheight(self):
    best, st = 0, [(self.root, 1)]
    while st:
        n, d = st.pop()
        ifnisself.NIL:
            continue
        best = max(best, d)
        st.append((n.left, d + 1))
        st.append((n.right,d+1))
    returnbest

defcheck(self):
    """Verify the five red-black properties; returns black-height."""
    assert self.root.color == BLACK, "root must be black"

    defrec(n):
        ifnisself.NIL:
            return 1
        ifn.color==RED:
            assert n.left.color == BLACK and n.right.color == BLACK, "red node with red child"
        lh, rh = rec(n.left), rec(n.right)
        assert lh == rh, "black-height mismatch"
        return lh + (1 if n.color == BLACK else 0)
    import sys
    sys.setrecursionlimit(10000)
    return rec(self.root)
```

4. B-Tree and 2-3 Tree

Properties

- B-tree of order *m*: every node has at most m−1 keys; every non-root node has at least ⌈m/2⌉−1 keys; an internal node with k keys has k+1 children; all leaves are at the same depth.
- A **2-3 tree** is exactly the case m=3 (1–2 keys, 2–3 children). The implementation is generic: `BTree(3)` is a 2-3 tree.
- The tree grows or shrinks only at the root, so it is perfectly height-balanced.

Operations

Operation	Steps
Search	In each node binary-search the keys; follow the matching child pointer.
Insert	Insert the key in the correct leaf. If the node has more than m−1 keys it **splits**: the middle key moves up to the parent, the two halves become siblings. Splits may propagate to the root, increasing the height by one.
Delete	Internal key: replace it by its in-order predecessor and delete that from the leaf. If a node falls below the minimum: **borrow** a key through the parent from a sibling that has a spare key, otherwise **merge** with a sibling and pull the separating key down. Merges may propagate upward; an empty root is removed (height decreases).

Source code

```
frombisectimportbisect_left


classBNode:
    slots=("keys","children")

    definit(self):
        self.keys,self.children=[],[]

    @property
    defleaf(self):
        returnnotself.children


classBTree:
    """maxkeyspernode=m-1,minkeyspernon-rootnode=ceil(m/2)-1."""

    def init(self, order=3): assert
        order >= 3
        self.m = order
        self.max_keys=order-1
        self.min_keys = (order + 1) // 2 - 1
        self.root = BNode()
        self.splits=self.merges=0

    defsearch(self,key): n
        = self.root while
        True:
            i=bisect_left(n.keys,key)
            if i < len(n.keys) and n.keys[i] == key:
                return True
            ifn.leaf:
                return False
            n=n.children[i]

    # ---------------------- insertion ----------------------
    def insert(self, key):
        ifself.search(key):
            returnFalse
        res=self._insert(self.root,key)
        if res:                                         # root split -> tree grows in height
            mid, right = res
            new_root = BNode()
            new_root.keys=[mid]
            new_root.children = [self.root, right]
            self.root = new_root
        returnTrue

    def_insert(self,node,key):
        i = bisect_left(node.keys, key)
        if node.leaf:
            node.keys.insert(i,key)
        else:
            res = self._insert(node.children[i], key)
            if res:
                mid, right = res
                node.keys.insert(i, mid)
                node.children.insert(i + 1, right)
        if len(node.keys) > self.max_keys:              #overflow->split
            self.splits += 1
            h=len(node.keys)//2 mid
            = node.keys[h] right =
            BNode()
            right.keys = node.keys[h + 1:]
            node.keys = node.keys[:h]
            ifnotnode.leaf:
                right.children = node.children[h + 1:]
                node.children = node.children[:h + 1]
            returnmid,right
        return None

    # ---------------------- deletion ----------------------
    defdelete(self,key):
        ifnotself.search(key):
```

```
        return False
    self._delete(self.root, key)
    if not self.root.keys and self.root.children:# root empty -> height shrinks
        self.root = self.root.children[0]
    returnTrue

def_delete(self,node,key):
    i=bisect_left(node.keys,key)
    if i < len(node.keys) and node.keys[i] == key:
        if node.leaf:
            node.keys.pop(i)
            return
        pred = self._max_key(node.children[i])       #replacebypredecessor
        node.keys[i] = pred
        self._delete(node.children[i], pred)
    else:
        self._delete(node.children[i], key)
    self._fix_child(node, i)

def_max_key(self,n): while
    not n.leaf:
        n=n.children[-1]
    return n.keys[-1]

def _fix_child(self, node, i):
    child = node.children[i]
    if len(child.keys) >= self.min_keys:
        return
    if i > 0 and len(node.children[i - 1].keys) > self.min_keys:      #borrowfromleftsibling
        left = node.children[i - 1]
        child.keys.insert(0, node.keys[i - 1])
        node.keys[i - 1] = left.keys.pop()
        ifnotleft.leaf:
            child.children.insert(0,left.children.pop())
    elif i < len(node.children) - 1 and len(node.children[i + 1].keys) >self.min_keys: #
        borrow from right sibling
        right = node.children[i + 1]
        child.keys.append(node.keys[i])
        node.keys[i] = right.keys.pop(0)
        if not right.leaf:
            child.children.append(right.children.pop(0))
    else:                                           #mergewithasibling
        self.merges+=1 if
        i > 0:
            i-=1
        left, right = node.children[i], node.children[i + 1]
        left.keys.append(node.keys.pop(i))
        left.keys += right.keys
        left.children += right.children
        node.children.pop(i + 1)

# ---------------------- helpers ----------------------
definorder(self):
    out = []

    defrec(n):
        for j, k in enumerate(n.keys):
            if not n.leaf:
                rec(n.children[j])
            out.append(k)
        if not n.leaf:
            rec(n.children[-1])
    rec(self.root)
    return out

defheight(self):
    h,n=1,self.root while
    not n.leaf:
        n=n.children[0] h
        += 1
    returnh

def check(self):
    depths=set()

    defrec(n,d,is_root):
        assertn.keys==sorted(n.keys)
```

```
            assert len(n.keys) <= self.max_keys
            if not is_root:
                assert len(n.keys) >= self.min_keys, "underflow"
            if n.leaf:
                depths.add(d)
            else:
                assert len(n.children) == len(n.keys) + 1
                for c in n.children:
                    rec(c,d+1,False)
        rec(self.root, 1, True)
        assertlen(depths)<=1,"leavesatdifferentdepths"

    defshow(self):
        lvl,out=[self.root],[] while
        lvl:
            out.append("".join("[" + "".join(map(str, n.keys)) + "]" for n in lvl)) lvl
            = [c for n in lvl for c in n.children]
        returnout
```

5. Splay Tree

Properties

- Nocolourorheightfieldisstored;thestructureadaptsitself.After*every*access(search,insertor delete) the accessed node is brought to the root by splaying.
- Splay steps: **zig** (one rotation), **zig-zig** (two rotations in the same direction), **zig-zag** (two rotations inopposite directions).
- AmortizedcostisO(logn)peroperation(potential-functionargument),butoneoperationcantake O(n).
- Frequentlyaccessedkeysstayneartheroot,sosplaytreesperformwellwhenaccessesareskewedor have locality.

Operations

Operation	Steps
Search(k)	Splay k (or the last node visited if k is absent) to the root.
Insert(k)	Splay k; if absent, split the tree at the root into keys < k and > k and make k the new root.
Delete(k)	Splayktotheroot;removeit;splaythemaximumoftheleftsubtreetoitsrootandattachtheright subtree.

Source code (top-down splaying)

```
classSNode:
    slots=("key","left","right")

    definit(self,key):
        self.key,self.left,self.right=key,None,None


classSplayTree:
    def init(self):
        self.root=None

    def_splay(self,t,key): if
        t is None:
            returnNone
        N=SNode(None) l
        = r = N
        whileTrue:
            ifkey<t.key:
                ift.leftisNone:
                    break
                if key < t.left.key:                    #zig-zig:rotateright y
                    = t.left
                    t.left, y.right = y.right, t
                    t = y
                    ift.leftisNone: break
                r.left=t                                #linkright
                r,t=t,t.left elif
            key > t.key:
                ift.rightisNone:
                    break
                if key > t.right.key:                   #zig-zig:rotateleft y
                    = t.right
                    t.right,y.left=y.left,t t =
                    y
                    ift.rightisNone:
                        break
                l.right=t                               #linkleft
                l,t=t,t.right else:
                break
        l.right, r.left = t.left, t.right               #assemble
        t.left, t.right = N.right, N.left
        returnt

    defsearch(self,key):
        self.root = self._splay(self.root, key)          # accessed key moves to the root
        return self.root is not None and self.root.key == key

    definsert(self,key):
        if self.root is None:
            self.root=SNode(key)
            return True
        self.root = self._splay(self.root, key)
        if self.root.key == key:
            returnFalse
        n = SNode(key)
        ifkey<self.root.key:
            n.left, n.right, self.root.left = self.root.left, self.root,
        Noneelse:
            n.right, n.left, self.root.right = self.root.right, self.root, None
        self.root = n
        returnTrue

    defdelete(self,key):
        ifself.rootisNone:
            return False
        self.root = self._splay(self.root, key)
        if self.root.key != key:
            returnFalse
        if self.root.left is None:
            self.root=self.root.right
        else:
            right=self.root.right
```

```
            self.root = self._splay(self.root.left, key)    # max of left subtree becomes root
            self.root.right = right
        returnTrue

    definorder(self):
        out, st, x = [], [], self.root
        while st or x:
            whilex:
                st.append(x)
                x = x.left
            x = st.pop()
            out.append(x.key)
            x = x.right
        returnout

    defheight(self):
        best, st = 0, [(self.root, 1)] if self.root else []
        while st:
            n, d = st.pop()
            best=max(best,d)
            forcin(n.left,n.right): if
                c:
                    st.append((c,d+1))
        returnbest
```

6. Demonstration of Insertions and Deletions

The following is the actual output of the demonstration program.

```
--- RED-BLACK TREE (R = red, B = black; each line is one level) ---
insert(10) -> rotations so far: 0
insert(20) -> rotations so far: 0
insert(30) -> rotations so far: 1
insert(15) -> rotations so far: 1
insert(25) -> rotations so far: 1
insert( 5) -> rotations so far: 1
insert(1)->rotationssofar:1
    20B
    10R30B
    5B15B25R
    1R
black-height=3|height=4
delete(20) -> 25B | 10R30B | 5B15B | 1R | valid, black-height = 3
delete(5) -> 25B | 10R30B | 1B15B | valid, black-height = 3

--- 2-3 TREE (B-tree of order 3; every node has 1 or 2 keys) ---
insert 1..10 (height 3 , splits = 5 )
    [4]
    [2][68]
    [1][3][5][7][910]
afterdelete4,7,1(merges=1,height3)
    [6]
    [3][9]
    [2][5][8][10]

--- B-TREE of order 5 (2 to 4 keys per node) ---
    [6 10 20]
    [35][789][1217][25304050]

---SPLAYTREE---
insert 1..7 in sorted order -> root = 7 | height = 7
    7
    6
    5
    4
    3
    2
    1
search(3) -> 3 becomes the root; height = 4
    3
    26
    147
    5
delete(3)->inorder:[1,2,4,5,6,7]|root=2
```

Points to notice: inserting 10, 20, 30 triggers one rotation (a line-shaped violation); after inserting 1 a recolouring(case1)keepsthetreevalid;deletingblacknode5requiredrecolouringtorestorethe black-height. In the 2-3 tree, 5 splits occurred while inserting 1..10 and one merge occurred while deleting 4, 7 and 1. The sorted insertion made the splay tree a chain of height 7, and a single search(3) reduced the height to 4.

7. Testing

Everystructurewas tested against Python's built-in `set`with 12,000 random insert/delete operations (key range 0–3999). Return values of each operation were compared with the reference; the red-black properties and the B-tree invariants (key counts, sorted keys, equal leaf depth) were verified every 200 operations; finally 300 searches were checked and all keys were deleted. Result:

```
PASSRedBlackopsok,finalsize0
PASS BTree m=3 ops ok, final size 0       (thisisthe2-3tree)
PASS BTree m=4 ops ok, final size 0
PASS BTree m=5 ops ok, final size 0
PASS BTree m=6 ops ok, final size 0
PASS BTree m=8 ops ok, final size 0
PASS BTree m=16 ops ok, final size 0
PASS Splay ops ok, final size 0
```

8. Performance Analysis

Insertion of n = 20000 keys

Keys 0..19999 were inserted once in random order and once in sorted order ($\log_2 n$ = 14.3). Times are wall-clock milliseconds of this Python implementation and are only indicative; heights and counts are exact.

Structure	Height (random)	Time ms (random)	Rebalancing work(random)	Height (sorted)	Timems (sorted)	Rebalancing work (sorted)
Red-Black tree	18	36	11822 rotations	26	42	19974 rotations
2-3 tree (B-tree m=3)	12	114	14898 splits	14	159	19981 splits
B-tree m=16	4	41	1946 splits	5	30	2495 splits
Splay tree	49	53	-	20000	55	-



Figure4:Heightafterinserting20000keys.Balancedtreesareunaffectedbysortedinput;thesplaytreedegeneratesintoa chain.

- Red-blackandB-treesstaylogarithmicforbothorders:thered-blackheight18(random)and26 (sorted) are within the bound $2 \cdot \log_2(n+1) \approx 28.6$.

- The2-3treeisshorterthanthered-blacktree(height12–14)buteachnodeisheavier;aB-treewithm = 16 has height only 4–5, which is why large-order B-trees are used for disk indexes.
- The splay tree has no balance guarantee: sorted insertion produces height 20000. This is allowed by the amortized analysis – the cost is paid back by later accesses – but a single access can be O(n).

Deleting half of the keys

Structure	Height afterwards	Time ms	Verification result
Red-Black tree	17	24	valid RB tree, 15598 total rotations
2-3 tree (B-tree m=3)	11	104	valid B-tree, 7190 merges
B-tree m=16	4	36	valid B-tree, 800 merges
Splay tree	34	47	BST order preserved

After 10000 random deletions the red-black tree and the B-trees were verified to satisfy all structural invariants and the remaining keys were identical to the expected set.

Skewed versus uniform access (when do splay trees win?)

A red-black tree and a splay tree each held 5000 keys. In the **skewed** workload, 90% of 20000 searches were directed at 1% of the keys; in the **uniform** workload keys were chosen at random. The table gives the average depth (number of nodes on the path) of the accessed key.

Workload	Red-Black tree	Splay tree
Skewed (90% of accesses to 1% of keys)	11.5	8.0
Uniform random	11.6	15.9

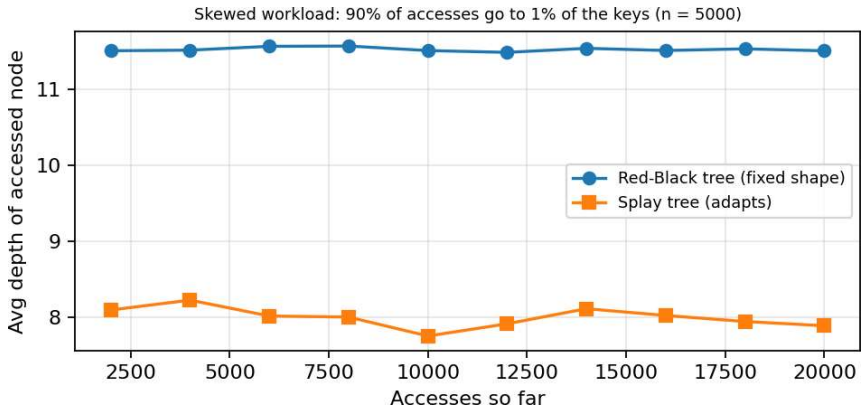


Figure 5: Under a skewed workload the splay tree keeps the hot keys near the root; the red-black tree's cost is fixed.

The splay tree is cheaper when accesses are concentrated on a few keys, but about 37% deeper on uniform random accesses, because every access restructures the tree without need.

9. Comparison and Selection Guide

Criterion	Red-Black	2-3 tree / B-tree	Splay
Worst-case search	O(log n)	O(log n)	O(n)single;O(logn) amortized
Balance info stored	1 bit per node	None (node sizes)	None
Rebalancingper update	≤2(insert)/3(delete) rotations	Splits / borrows / merges	Splaying after every access
Search modifies tree?	No	No	Yes (not suitable for read-only/concurrent readers)
Best suited for	In-memoryorderedmaps (e.g. C++ std::map, Java TreeMap)	Disk-based indexes, databases,filesystems	Caches,skewedorrepetitive access patterns
Implementation effort	High (many cases)	High for deletion	Low

10. Conclusion

- Red-blacktrees,2-3treesandB-treeswereimplementedwithinsertionanddeletionandpassed randomized tests with invariant checking.
- Red-blacktreeskeepheightwithin2·$\log_2$(n+1)(18and26measuredfor20000keys)usingfew rotations.
- B-trees and 2-3 trees are perfectly balanced; larger node orders greatly reduce height (m = 16: height 4–5), which suits block storage.
- Splay trees need no balance data and win on skewed access (average depth 8.0 vs 11.5) but lose on uniform access (15.9 vs 11.6) and have no worst-case guarantee per operation.
- Thechoiceofstructuredependsontheworkload:guaranteedbounds(red-black),storage-aware (B-tree) or adaptive (splay).

References

- T. H. Cormen, C. E. Leiserson, R. L. Rivest, C. Stein, *Introduction to Algorithms*, MIT Press (red-black trees, B-trees).
- D. D. Sleator, R. E. Tarjan, “Self-adjusting binary search trees”, *Journal of the ACM*, 32(3), 1985.
- R. Bayer, E. McCreight, “Organization and maintenance of large ordered indexes”, *Acta Informatica*,1972.
- M. A. Weiss, *Data Structures and Algorithm Analysis in C++*, 2nd ed., Pearson, 2004.
- M. T. Goodrich, R. Tamassia, *Algorithm Design*, John Wiley, 2002.